Unit–4: Vedic Mathematics Sutras

4.1 Nikhilam Navatashcaramam Dashatah

Meaning: “All from 9 and the last from 10.”

Concept / Definition

A fast multiplication method when numbers are **near a power of 10** (10, 100, 1000…).
You use:

- **Base** = nearest power of 10
- **Deficiency/Excess** = how far the number is from the base
- **Left part** = cross subtraction
- **Right part** = product of deficiencies (written with proper digits)

Where used

- Very fast for numbers close to **100** (like 97, 98, 102, 104 etc.)
- Also used for subtraction tricks near 10/100

Worked Example (98 × 97)

Base = 100
Deficiencies:

- 98 = 100 − 2 → −2
- 97 = 100 − 3 → −3

Steps:

1. Left part: 98 − 3 = **95** (or 97 − 2 = 95)
2. Right part: (2 × 3) = **6**
   Since base is 100 (two zeros), write right part in **2 digits** → **06**
3. **Answer = 9506**

Explanation

Both numbers are just a little less than 100. Subtract the "shortage" crosswise for the left part, multiply shortages for the right part.

Revision points

- Base 10 → right part 1 digit
- Base 100 → right part 2 digits
- Base 1000 → right part 3 digits

4.2 Ekadhikena Purvena

Meaning: "By one more than the previous one."

Concept / Definition

Most common use in basic syllabus: **square of numbers ending in 5**.
Rule: If number = **(10a + 5)**, then
[
(10a+5)^2 = a(a+1)\ \text{followed by}\ 25
]

Where used

- Quick squaring of numbers like 15, 25, 35, 45, 65, 95

Worked Example (65^2)

a = 6
6 × (6+1) = 6×7 = **42**
Attach **25** → **4225**

Answer 65^2 = 4225

Explanation

Take the first part (6), multiply with the next number (7), and put 25 at the end.

Unit–4: Vedic Mathematics Sutras

Revision points

- Always ends with **25**
- Only works directly for numbers ending in **5**

4.3 Udharan (Extraction)

Meaning: "The extraction."

Concept / Definition

Used for **extracting values** (like square roots) using known squares/factors/patterns.

Where used

- Finding **square roots of perfect squares** quickly
- Simplifying by recognizing known patterns

Worked Example (√144)

We know: 12×12 = 144
√144 = 12

Explanation

"Extraction" means pulling out the root by recognizing a perfect square.

Revision points

Memorize squares: 1^2 to 20^2 (helps a lot).

4.4 Paraavartya (Transposition and cancellation)

Meaning: "Transposition and cancellation."

Concept / Definition

Solve by **moving terms** from one side to another (transposition) and then **cancelling** common factors.
In basic use, it matches the common algebra steps used to solve equations.

Where used

- Solving linear equations quickly
- Simplifying expressions

Worked Example (Solve 3x + 5 = 20)

Transpose 5: $3x = 20 - 5 = 15$
Cancel 3: $x = 15/3 =$ **5**
✅ **x = 5**

Explanation

Move the extra number to the other side (sign changes), then divide to cancel.

Revision points

- Move across "=" → sign changes
- Cancel common factor at the end

4.5 Shunyam Saamyasamuccaye

Meaning: "When the sum is the same, that sum is zero."

Concept / Definition

If the **same expression** appears on both sides, subtract it and it becomes **0**, making the equation easy.

Where used

- Simplifying equations by removing equal parts from both sides

Worked Example (x + 7 = 10 − x)

Bring like terms together:
$x + x = 10 - 7$
$2x = 3$
$x = 3/2 =$ **1.5**
✅ **x = 1.5**

Explanation

Remove common parts first; solve what remains.

Revision points

Always try cancelling same terms on both sides before solving.

4.6 Anurupyena

Meaning: "Proportionately."

Concept / Definition

Use proportional adjustment (factors) to make multiplication easier.

Where used

- Multiplying with 25, 50, 125, etc.
- When one number can be converted to a fraction of 100 or 1000

Worked Example (16 × 25)

25 = 100/4
16×25 = 16×(100/4) = (16/4)×100 = 4×100 = **400**
✅ **Answer = 400**

Explanation

Convert 25 into 100/4, divide first, then multiply.

Revision points

Good pairs:

- ×25 = ÷4 then ×100
- ×50 = ÷2 then ×100

4.7 Sankalana–Vyavakalanabhyam

Meaning: "By addition and by subtraction."

Concept / Definition

Use algebra identity to avoid long multiplication, especially:
[
(a+b)(a-b)=a^2-b^2
]

Where used

- Multiply numbers close to 100: 103×97, 104×96, etc.

Worked Example (103 × 97)

$(100+3)(100-3) = 100^2 - 3^2$
$= 10000 - 9 =$ **9991**
✓ **Answer = 9991**

Explanation

Instead of multiplying fully, do "square minus square".

Revision points

If numbers are equally away from a base, use $a^2 - b^2$.

4.8 Puranapuranabhyam

Meaning: "By the completion or non-completion."

Concept / Definition

Complete a number to a round base (10/100), perform easy operation, then adjust back.

Where used

- Addition/subtraction near 10/100
- Quick mental calculations

Worked Example (99 + 37)

$99 = 100 - 1$
$(100 - 1) + 37 = 137 - 1 =$ **136**
✓ **Answer = 136**

Explanation

Make 99 into 100, add, then subtract 1.

Revision points

Complete to nearest round number whenever possible.

4.9 Chalana–Kalana

Meaning: "By motion or by applying a shift."

Concept / Definition

Shift place value (move zeros/decimal shifts) to simplify calculations.

Where used

- Multiplication with 10, 100, 1000
- Numbers like 30×14, 120×7, etc.

Worked Example (30 × 14)

30 = 3×10
30×14 = (3×14)×10 = 42×10 = **420**
✅ **Answer = 420**

Explanation

Do the small multiplication first, then attach the zero(s).

Revision points

Shifting is essentially place-value multiplication.

4.10 Yavadunam

Meaning: "Whatever is the deficiency."

Concept / Definition

For multiplying numbers **below a base** (commonly 10). Use deficiencies and combine.

Where used

- Fast multiplication for 6–9 range (base 10)

Worked Example (8 × 7)

Base = 10
Deficiencies: 8 is −2, 7 is −3
Left: 8 − 3 = 5 (or 7 − 2 = 5)
Right: 2×3 = 6 (1 digit for base 10)
✅ **Answer = 56**

Explanation

Subtract crosswise, multiply deficiencies.

Revision points

Base 10 → right side 1 digit always.

4.11 Vyastisamanstih

Meaning: "The parts and the whole."

Concept / Definition

Relates **parts** with the **whole**. If whole is known and one part is known, the other part is found by subtraction.

Where used

- Word problems, partition problems
- Basic algebra reasoning

Worked Example

Total = 60, Part A = 25
Part B = 60 − 25 = **35**
✅ **Other part = 35**

Explanation

Whole = sum of parts; missing part = whole − known part.

Revision points

Very common in basic “total and remaining” sums.

4.12 Sesanyan

Meaning: “The remainder.”

Concept / Definition

Focuses on problems involving **remainders** after division.

Where used

- Remainder problems
- Quick checks in division

Worked Example (Remainder of 29 ÷ 5)

$5 \times 5 = 25$
Remainder = 29 − 25 = **4**
✅ **Remainder = 4**

Explanation

After taking the biggest multiple, whatever is left is the remainder.

Revision points

Remainder is always less than the divisor.

4.13 Gunitasamuchyah

Meaning: “The product of the sum.”

Concept / Definition

A quick **verification/checking** method using digit sums (commonly taught as a quick consistency check).

Where used

- Checking multiplication results quickly in exams

Worked Example (Check 23×14 = 322)

Digit sum:

- 23 → 2+3 = 5
- 14 → 1+4 = 5
  Product: 5×5 = 25 → digit sum 2+5 = 7
  Answer digit sum: 322 → 3+2+2 = 7 ✅
  So result is consistent.

Explanation

If digit-sum check matches, your answer is likely correct (good for catching many mistakes).

Revision points

Not a full proof, but a strong quick check.

4.14 Vistaran

Meaning: "Expansion."

Concept / Definition

Use algebraic expansion to multiply:
[
(a+b)(c+d)=ac+ad+bc+bd
]

Where used

- Multiplication like 12×13, 27×13 etc.
- Algebra expansion

Worked Example (12 × 13)

(10+2)(10+3)
= 100 + 30 + 20 + 6
= **156**
✅ **Answer = 156**

Explanation

Break into tens and ones, multiply parts, add all.

Revision points

This is distributive property in a clean form.

4.15 Rupan

Meaning: "Form."

Concept / Definition

Convert a problem into a simpler **form** (percentage → fraction, complex → simple) for quick solving.

Where used

- Percent problems
- Ratio/fraction simplification

Worked Example (50% of 36)

50% = 1/2
(1/2)×36 = **18**
✅ **Answer = 18**

Explanation

Change to a friendlier format and solve easily.

Revision points

Common conversions:

- 50% = 1/2
- 25% = 1/4
- 10% = 1/10

4.16 Chidana

Unit–4: Vedic Mathematics Sutras

Meaning: “By splitting.”

Concept / Definition

Split a number into convenient parts and use distributive property:
[
(a+b)c = ac+bc
]

Where used

- Fast mental multiplication like 48×6, 54×7, etc.

Worked Example (48 × 6)

48 = 40 + 8
40×6 = 240
8×6 = 48
Total = 240 + 48 = **288**
✅ **Answer = 288**

Explanation

Break the big number into easy chunks, multiply each chunk, add.

Revision points

Always split into tens + ones for speed.

Practice Questions

A. Do as Directed (1–2 marks)

1. Use Ekadhikena: 35^2
2. Use Anurupyena: 24×25
3. Find remainder (Sesanyan): 47 ÷ 6
4. Use completion: 99 + 58
5. Solve using transposition: 4x − 7 = 21

B. Do as Directed (3–5 marks)

6. Use Nikhilam: 96×98
7. Use Yavadunam: 9×8

8. Use (a+b)(a−b): 104×96
9. Use Vistaran: 27×13
10. Use Chidana: 54×7
11. Parts & whole: Total = 120, one part = 47, find other part
12. Check using Gunitasamuchyah: Is 18×17 = 306 correct?

Answer Key

1. 35^2 = **1225**
2. 24×25 = **600** (24×100/4 = 6×100)
3. **5**
4. **157**
5. x = **7**
6. 96×98 = **9408**
7. 9×8 = **72**
8. 104×96 = **9984**
9. 27×13 = **351**
10. 54×7 = **378**
11. 120−47 = **73**
12. **Yes**, consistent by digit-sum check

PYTHON IMPLEMENTATION OF EACH SUTRA

```
# -----------------------------------------------------------
# 4.1 Nikhilam (All from 9 and last from 10) - Near Base Multiply
# -----------------------------------------------------------
def nikhilam_multiply(a: int, b: int, base: int = None) -> int:
    """
    Multiply two numbers near a base (10, 100, 1000).
    If base not given, choose 10/100/1000 based on max(a,b).

    Example: 98*97 with base 100.
    """
    if base is None:
        m = max(a, b)
        if m < 10:
            base = 10
        elif m < 100:
            base = 100
        elif m < 1000:
```

```
        base = 1000
    else:
        # For larger, pick next power of 10
        base = 10 ** (len(str(m)))

    da = a - base
    db = b - base

    left = a + db  # same as b + da
    right = da * db

    # number of digits on right side equals zeros in base
    digits = len(str(base)) - 1

    # handle negative right part (rare in basic syllabus; occurs if one above base and one
below)
    if right >= 0:
        right_str = str(right).zfill(digits)
        return int(str(left) + right_str)
    else:
        # If right is negative, borrow 1 from left and adjust right
        left -= 1
        right = base + right
        right_str = str(right).zfill(digits)
        return int(str(left) + right_str)


# ------------------------------------------------------------
# 4.2 Ekadhikena Purvena - Square numbers ending with 5
# ------------------------------------------------------------
def ekadhikena_square_ending_5(n: int) -> int:
    """
    Square of a number ending in 5 using:
    (10a+5)^2 = a(a+1) followed by 25
    """
    if n % 10 != 5:
        raise ValueError("This method is for numbers ending in 5 only.")
    a = n // 10
    left = a * (a + 1)
```

```
    return int(str(left) + "25")


# -----------------------------------------------------------
# 4.3 Udharan (Extraction) - Integer square root extraction (perfect squares)
# -----------------------------------------------------------
def udharan_sqrt_perfect_square(n: int) -> int:
    """
    Extract square root for perfect squares (basic classroom version).
    Returns root if perfect square, else raises ValueError.
    """
    if n < 0:
        raise ValueError("Negative numbers do not have real square roots.")

    x = 0
    while x * x < n:
        x += 1

    if x * x == n:
        return x
    raise ValueError("Not a perfect square (basic Udharan version).")


# -----------------------------------------------------------
# 4.4 Paraavartya (Transposition and cancellation) - Solve ax + b = c
# -----------------------------------------------------------
def paraavartya_solve_linear(a: int, b: int, c: int) -> float:
    """
    Solve ax + b = c by transposition & cancellation:
    x = (c - b) / a
    """
    if a == 0:
        raise ValueError("a cannot be 0 in ax + b = c.")
    return (c - b) / a


# -----------------------------------------------------------
# 4.5 Shunyam Saamyasamuccaye - Cancel common terms: (x + p) = (q - x)
# -----------------------------------------------------------
```

```
def shunyam_solve_x_plus_p_eq_q_minus_x(p: float, q: float) -> float:
    """
    Solve: x + p = q - x
    Combine like terms (cancel common structure idea):
    2x = q - p  => x = (q - p)/2
    """
    return (q - p) / 2


# ------------------------------------------------------------
# 4.6 Anurupyena (Proportionately) - Multiply by 25 quickly
# ------------------------------------------------------------
def anurupyena_times_25(n: int) -> int:
    """
    n * 25 = (n / 4) * 100, works when n divisible by 4 for integer result.
    For other n, returns exact integer via normal multiply (still correct).
    """
    if n % 4 == 0:
        return (n // 4) * 100
    return n * 25


# ------------------------------------------------------------
# 4.7 Sankalana-Vyavakalanabhyam - (a+b)(a-b) = a^2 - b^2
# ------------------------------------------------------------
def sankalana_multiply_symmetric(a: int, b: int) -> int:
    """
    Multiply (a+b)(a-b) quickly:
    returns a^2 - b^2
    Example: (100+3)(100-3) => a=100, b=3
    """
    return a * a - b * b


# ------------------------------------------------------------
# 4.8 Puranapuranabhyam - Completion (near round number) addition
# ------------------------------------------------------------
def purana_add_near_base(x: int, y: int, base: int = 100) -> int:
    """
```

```
    Add x + y by completing x to a base:
    if x is near base (like 99), do (base - d) + y = base + y - d
    """
    d = base - x
    return base + y - d


# ----------------------------------------------------------
# 4.9 Chalana-Kalana - Shift-based multiplication by powers of 10
# ----------------------------------------------------------
def chalana_kalana_shift_multiply(n: int, multiplier: int) -> int:
    """
    Multiply using shifting when multiplier is 10, 100, 1000...
    Example: 14*30 => treat 30 as 3*10, shift by 10.
    """
    # if multiplier is power of 10
    m = multiplier
    zeros = 0
    while m % 10 == 0 and m > 1:
        zeros += 1
        m //= 10

    # now multiplier = m * (10^zeros)
    return (n * m) * (10 ** zeros)


# ----------------------------------------------------------
# 4.10 Yavadunam - Multiply numbers below base 10 (basic use)
# ----------------------------------------------------------
def yavadunam_multiply(a: int, b: int, base: int = 10) -> int:
    """
    Multiply two numbers below a base (commonly 10).
    Same mechanism as Nikhilam but simplest for base 10.

    Example: 8*7 => (8-3)(2*3) => 56
    """
    if not (a < base and b < base):
        raise ValueError("This basic function assumes both numbers are below the base.")
    da = base - a
```

```
    db = base - b

    left = a - db  # or b - da
    right = da * db

    digits = len(str(base)) - 1  # base 10 -> 1 digit
    right_str = str(right).zfill(digits)
    return int(str(left) + right_str)


# ----------------------------------------------------------
# 4.11 Vyastisamanstih - Parts and whole
# ----------------------------------------------------------
def vyasti_find_other_part(total: int, part: int) -> int:
    """
    If total and one part is given, other part = total - part
    """
    return total - part


# ----------------------------------------------------------
# 4.12 Sesanyan - Remainder
# ----------------------------------------------------------
def sesanyan_remainder(n: int, d: int) -> int:
    """
    Return remainder when n is divided by d.
    """
    if d == 0:
        raise ValueError("Divisor cannot be 0.")
    return n % d


# ----------------------------------------------------------
# 4.13 Gunitasamuchyah - Digit-sum check for multiplication
# ----------------------------------------------------------
def digit_sum(n: int) -> int:
    n = abs(n)
    s = 0
    while n > 0:
```

```
        s += n % 10
        n //= 10
    return s

def digital_root(n: int) -> int:
    """
    Digital root (1-9), with 0 staying 0.
    Used for quick consistency checks.
    """
    if n == 0:
        return 0
    n = abs(n)
    while n >= 10:
        n = digit_sum(n)
    return n

def gunitasamuchyah_check(a: int, b: int, claimed: int) -> bool:
    """
    Check if a*b is consistent with claimed using digital root check.
    Returns True if consistent (not a proof), else False.
    """
    left = digital_root(a) * digital_root(b)
    left = digital_root(left)
    right = digital_root(claimed)
    return left == right


# ------------------------------------------------------------
# 4.14 Vistaran - Expansion multiplication (distributive)
# ------------------------------------------------------------
def vistaran_multiply_two_digit(x: int, y: int) -> int:
    """
    Multiply using expansion:
    Split into tens and ones (best for 2-digit numbers).
    """
    x_tens = (x // 10) * 10
    x_ones = x % 10
    y_tens = (y // 10) * 10
    y_ones = y % 10
```

```
    return x_tens * y_tens + x_tens * y_ones + x_ones * y_tens + x_ones * y_ones


# ------------------------------------------------------------
# 4.15 Rupan - Convert form (percent to fraction example)
# ------------------------------------------------------------
def rupan_percent_of(pct: float, value: float) -> float:
    """
    Convert percent to fraction and compute:
    pct% of value = (pct/100) * value
    """
    return (pct / 100.0) * value


# ------------------------------------------------------------
# 4.16 Chidana - Splitting multiplication
# ------------------------------------------------------------
def chidana_multiply_split(n: int, m: int) -> int:
    """
    Split n into convenient parts: tens + ones (simple approach).
    n*m = (tens*m) + (ones*m)
    """
    tens = (n // 10) * 10
    ones = n % 10
    return tens * m + ones * m


# ============================================================
# DEMO (Run this section)
# ============================================================
if __name__ == "__main__":
    print("4.1 Nikhilam 98*97 =", nikhilam_multiply(98, 97, 100))
    print("4.2 Ekadhikena 65^2 =", ekadhikena_square_ending_5(65))
    print("4.3 Udharan sqrt(144) =", udharan_sqrt_perfect_square(144))
    print("4.4 Paraavartya solve 3x+5=20 -> x =", paraavartya_solve_linear(3, 5, 20))
    print("4.5 Shunyam solve x+7=10-x -> x =", shunyam_solve_x_plus_p_eq_q_minus_x(7,
10))
    print("4.6 Anurupyena 16*25 =", anurupyena_times_25(16))
```

```
    print("4.7 Sankalana (100+3)(100-3) =", sankalana_multiply_symmetric(100, 3))
    print("4.8 Puranapuranabhyam 99+37 =", purana_add_near_base(99, 37, 100))
    print("4.9 Chalana-Kalana 14*30 =", chalana_kalana_shift_multiply(14, 30))
    print("4.10 Yavadunam 8*7 =", yavadunam_multiply(8, 7, 10))
    print("4.11 Vyasti total 60, part 25 -> other =", vyasti_find_other_part(60, 25))
    print("4.12 Sesanyan remainder 29%5 =", sesanyan_remainder(29, 5))
    print("4.13 Gunitasamuchyah check 23*14=322 ->", gunitasamuchyah_check(23, 14,
322))
    print("4.14 Vistaran 12*13 =", vistaran_multiply_two_digit(12, 13))
    print("4.15 Rupan 50% of 36 =", rupan_percent_of(50, 36))
    print("4.16 Chidana 48*6 =", chidana_multiply_split(48, 6))
```

OUTPUT:

```
4.1 Nikhilam 98*97 = 9506
4.2 Ekadhikena 65^2 = 4225
4.3 Udharan sqrt(144) = 12
4.4 Paraavartya solve 3x+5=20 -> x = 5.0
4.5 Shunyam solve x+7=10-x -> x = 1.5
4.6 Anurupyena 16*25 = 400
4.7 Sankalana (100+3)(100-3) = 9991
4.8 Puranapuranabhyam 99+37 = 136
4.9 Chalana-Kalana 14*30 = 420
4.10 Yavadunam 8*7 = 56
4.11 Vyasti total 60, part 25 -> other = 35
4.12 Sesanyan remainder 29%5 = 4
4.13 Gunitasamuchyah check 23*14=322 -> True
4.14 Vistaran 12*13 = 156
4.15 Rupan 50% of 36 = 18.0
4.16 Chidana 48*6 = 288
```